```
initialize = (teams...) ->
  insert teams...

initialize 'wolverines', 'sabertooths', 'mongooses'
# [ 'wolverines','sabertooths', 'mongooses' ]
```

Using rest and spread in combination, you can make a flexible and elegant program to highlight the team names in the table. This program appears in the following listing. Note that this listing is intended to run in a web browser; you can use the server from listing 7.5 to achieve that.

Listing 7.1. Competition

```
find = (name) ->
  document.querySelector ".#{name}"
```

The find function uses the native document.querySelector to find elements matching the class name.

```
color = (element, color) ->
  element.style.background = color
```

Set the background color of the element via a style property.

```
insert = (teams...) ->
  root = document.querySelector '.teams'
  for team in teams
    element = document.createElement 'li'
    element.innerHTML = team
    element.className = team
    root.appendChild element
```

Add elements to the page for each team.

```
highlight = (first, rest...) ->
  color find(first), 'gold'
  for name in rest
    color find(name), 'blue'
```

Highlight the first element gold and then the rest blue via a comprehension.

```
initialize = (ranked) ->
  insert ranked...
  first = ranked.slice(0, 1)
  rest = ranked.slice 1
  highlight first, rest...
```

Initialize by inserting the elements and then highlighting.

```
window.onload = ->
  initialize [
    'wolverines'
    'wildcats'
    'mongooses'
  ]
```

Invoke initialize with the array of teams when the browser window loads.

Although rest and spread are only small bits of syntactic sugar (remember, syntactic sugar makes things sweeter), they can make a big difference for code readability, which is a key motivation for many of the syntactic